Task B – Fox Algorithm **Subham Gaurav** **1564925**

Matrix Distribution:

Using **p = q × q** MPI processes **(where q = sqrt(p)),** each process is assigned a submatrix block of size **(n/q) × (n/q).**

- **Rank 0** generates full matrices A and B using **fillMatrix**().
- It extracts blocks using grid coordinates (**MPI_Cart_coords**) and sends them with **MPI_Send**.
- Each process receives one submatrix block from A and B.

Exchange & Computation (Fox Algorithm)

Each process computes a partial block of matrix C over q iterations:

- **Broadcast A Block** (**MPI_Bcast**):
  On each iteration, the row communicator broadcasts the A block from the pivot column **(my_row + step) % q**.
- **Block Multiplication**:
  Each process computes **C += A × B** using nested loops and accumulates into local C.
- **Cyclic Shift of B Block** (MPI_Cart_shift, MPI_Sendrecv_replace):
  B blocks are shifted **upward** across the process grid columns with wrap-around.
- **Result Gathering** (MPI_Send, MPI_Recv):
  Each process sends its computed C block to Rank 0, which reconstructs the full result matrix using received blocks and their coordinates.

Code Changes from Task A to Task B

To convert from **element-wise computation** in **Task A** to **submatrix-based computation** in **Task B**, I changed the code to work with **matrix blocks** instead of single values. Each process now receives a square block of elements from matrices A and B (instead of just one) and computes a full block of matrix C using nested loops. This required introducing **MPI_Cart_create** and **MPI_Cart_coords** to map processes into a **2D Cartesian grid** and calculate **submatrix positions**. Instead of replacing core MPI functions, I reused **MPI_Send**, **MPI_Recv**, and **MPI_Bcast** but adapted them to send and broadcast entire blocks. I also added **MPI_Cart_shift** for clean **cyclic shifting of B blocks** across the grid columns. The logic of the Fox algorithm remains the same—just the **granularity of data and computation** changed.